МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»

Лабораторная работа
по курсу «Информационный поиск»

Поисковой движок.

Выполнил: Андрюшин Л. Д.
Группа: М8О-412Б-22
Оценка:
Дата сдачи:
Преподаватели: Кухтичев А. А.

Москва, 2025

Добыча корпуса документов

Я собрал корпус из русскоязычной Википедии по категории «Литература» и вложенным подкатегориям. Очевидно, что внутри категориии «Литература» существует множество самых разнообразных подкатегорий. Так что условие на минимум 2 источника выполняется. Скачивание выполнялось скриптом, который обходит категорию с глубиной 4 и сохраняет информацию о странице в файл docs.jsonl. Он получает как сырой HTML, так и текстовую версию страницы, сохраняет их вместе с заголовками HTTP (Last-Modified, Etag (для проверки обновлений страницы в будущем)) и другими данными в файл docs.jsonl. Так же ведется отдельный файл processed.txt, чтобы помнить, какие страницы уже обработаны, поддерживает возобновление с места остановки, выводит прогресс загрузки и корректно обрабатывает ошибки или прерывания пользователем.

Предобработка

Далее идет файл obrabotka.py Этот скрипт берёт уже скачанные статьи из Википедии (JSONL файл docs.jsonl) и формирует из них структурированный корпус текста для анализа или обработки. Он поочерёдно читает каждую строку JSONL, проверяет наличие уникального ID статьи, пропускает дубликаты и слишком короткие статьи, а затем очищает текст: нормализует Unicode, заменяет специальные пробелы, приводя тире и дефисы к единому виду, удаляет управляющие символы, склеивает слова, разделённые переносом, и схлопывает лишние пробелы.

Очищенный текст вместе с ID и заголовком статьи сохраняется в папки с TSV-файлами (part_001.tsv, part_002.tsv и т.д.), по 1000 документов на файл. Скрипт ведёт лог обработанных и пропущенных статей (processing.log), учитывает уже обработанные документы, чтобы можно было дозапустить процесс, и собирает статистику: количество обработанных строк, уникальных документов, объём исходных и очищенных данных, количество дубликатов и слишком коротких статей.

В конце создаётся файл corpus_info.txt с полной информацией о корпусе, включая источники, размеры, количество файлов и дату создания, а также выводится пример первого документа. Скрипт позволяет превратить сырые данные Википедии в готовый к использованию корпус текстов с очищенным содержимым и удобным форматом для анализа. Вот его вывод:

```
ИНФОРМАЦИЯ О КОРПУСЕ
==================================================

ИСТОЧНИКИ:
1. https://ru.wikipedia.org/wiki/Категория:Литература
2. Подкатегории (глубина обхода: 4 уровня)
```

СТАТИСТИКА:
Всего обработано строк: 34546
Уникальных документов сохранено: 31258
Пропущено (нет ID): 0
Пропущено (дубликаты): 3286
Пропущено (слишком короткие): 2

РАЗМЕРЫ:
Сырые данные: 279.56 МБ
Очищенный текст: 239.74 МБ
Средний размер документа: 7.85 КБ

ФАЙЛЫ:
Количество TSV-файлов: 32
Документов в файле: до 1000
Формат: TSV (id, title, text)
Кодировка: UTF-8

ДАТА СОЗДАНИЯ: 2026-02-04 19:25:54

Загрузка данных в БД

Далее идет файл mongo_loader.py. Он последовательно читает каждую строку JSONL, нормализует URL страницы, конвертирует дату последнего получения в Unix timestamp и определяет источник документа, затем формирует структуру для MongoDB, включая сырой HTML, заголовок, ETag, дату последнего изменения и уникальный идентификатор документа. Скрипт проверяет, существует ли уже запись в базе по нормализованному URL, и в зависимости от того, изменился ли документ (сравнивая ETag, дату модификации или размер HTML), либо обновляет его, либо пропускает, либо добавляет как новый. В процессе ведётся статистика: количество обработанных, вставленных, обновлённых и пропущенных документов, а также объём данных, и выводится прогресс каждые 100 строк. Скрипт учитывает ошибки, JSON-декодирование и прерывания пользователем, поддерживает задержку между обработкой записей и создаёт необходимые индексы в коллекции для ускорения поиска по URL, ETag и времени обхода, обеспечивая эффективное поддержание базы актуальных страниц Википедии.

Токенизация

Файл tokenizer.cpp последовательно читает каждую статью, извлекает её ID, заголовок и текст, а затем разбивает текст на токены (слова и словосочетания с дефисами), приводя их к нижнему регистру. Скрипт собирает подробную статистику: общее количество документов и токенов, количество уникальных токенов, среднюю длину токена, длину

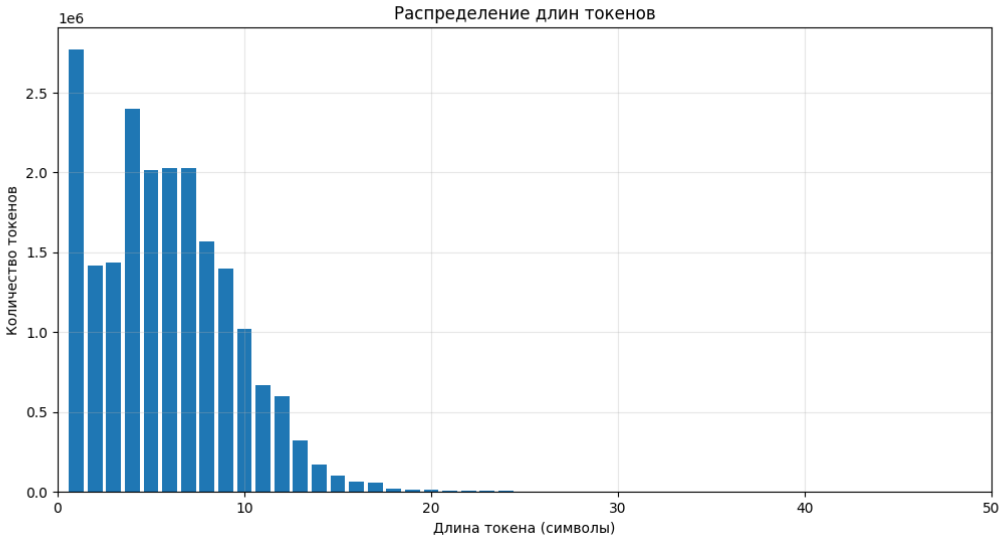
документов в токенах (среднюю, медианную, минимальную и максимальную) и суммарный объём текста в байтах. Он также подсчитывает частоты каждого токена, запоминает порядок их первого появления и формирует список редких токенов, встречающихся только один раз. Кроме того, скрипт определяет самый короткий и самый длинный документ по количеству токенов и сохраняет первые 300 символов текста этих документов для примера. В конце он выводит статистику на экран, включая скорость обработки (КБ/сек и токенов/сек), и записывает всю информацию в файл tokenization_stats.txt. Вот вывод данного файла:

```
СТАТИСТИКА ТОКЕНИЗАЦИИ
==================================================

ОБЩАЯ СТАТИСТИКА:
  Документов: 31258
  Объём текста: 239.74 МБ
  Объём текста: 245489.02 КБ

ТОКЕНЫ:
  Всего токенов: 20139876
  Уникальных токенов: 957333
  Средняя длина токена: 5.83

ДОКУМЕНТЫ:
  Средняя длина документа: 644 токенов
  Медианная длина документа: 408 токенов
  Минимальная длина: 26 токенов
  Максимальная длина: 32146 токенов

ПРОИЗВОДИТЕЛЬНОСТЬ:
  Время выполнения: 10.91 сек
  Скорость: 22504.13 КБ/сек
  Скорость: 1846235 токенов/сек

ТОП-20 САМЫХ ЧАСТЫХ ТОКЕНОВ:
  в: 966225
  и: 575721
  с: 298621
  на: 219723
  году: 140355
  года: 116049
  а: 113239
  по: 99410
  его: 96081
  м: 90046
  о: 80732
  к: 80349
  из: 80130
  был: 78851
  он: 73897
  не: 70637
  за: 58425
  как: 54556
  что: 52194
  г: 48189
```

ПРИМЕРЫ РЕДКИХ ТОКЕНОВ (первые 20):
eratura, проецируем, осуществивших, интервьюировавшей, несловесных, натяжка, исполнявшегося, авторов-дилетантов, отграничивающие, приблизительно-ориентировочный, разветвлённые, библиотечно-библиографической, словесно-языкового, стандартизованы, малоподготовленных, рецензируются, аттестуются, упорядочивающие, подразумеваемых, делящаяся

САМЫЙ МАЛЕНЬКИЙ ДОКУМЕНТ:
Название: Жоффруа Вандомский
Токенов: 26
Текст: Жоффруа Вандомский (лат. Goffridus Vindocinensis; 1065/1070, Анже, Франция - 26 марта 1132, там же) - бенедиктинский монах, писатель и кардинал. Его труды включены в «Patrologia Latina». == Примечания ==

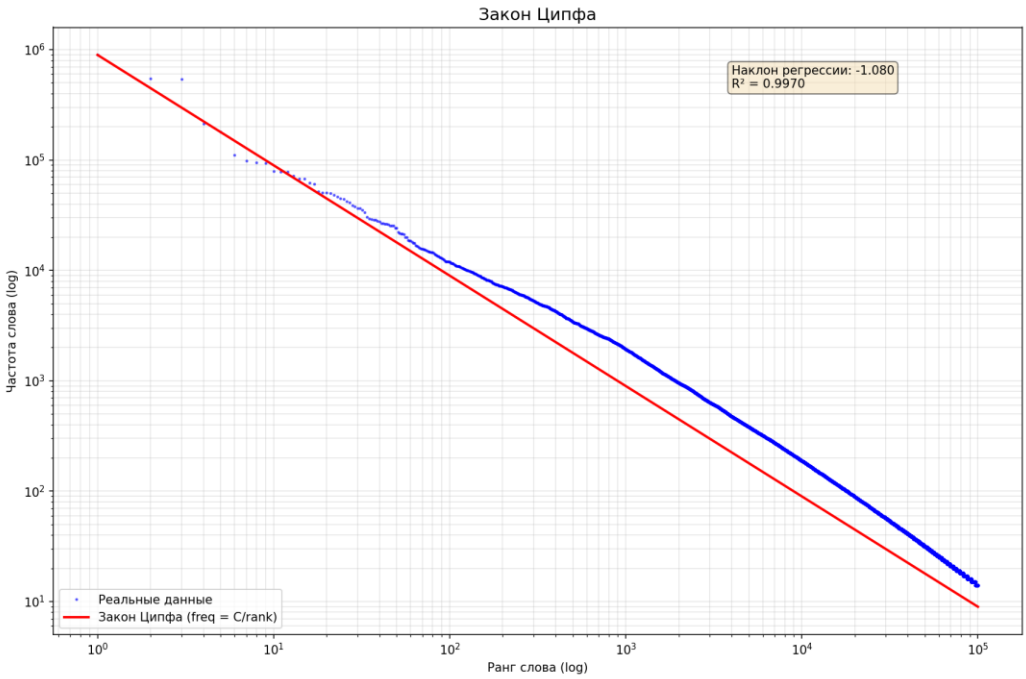
САМЫЙ БОЛЬШОЙ ДОКУМЕНТ:
Название: Бхактиведанта Свами Прабхупада
Токенов: 32146
Текст: Абха́й Чаранаравѝнда Бхактиведа́нта Сва́ми Прабхупа́да (санскр. अभयचरणारविन्द भक्तिवेदान्त स्वामीप्रभुपाद, IAST: abhaya-caraṇāravinda bhakti-vedānta svāmī prabhupāda, бенг. অভয় চরণারবিন্দ ভক্তিবেদান্ত স্বামী প্রভুপাদ, также известен как Шри́ла П р а б х у п а ́ д а ; 1 сентября 1896, Калькутта - 14 ноября 19...



Закон Ципфа

Следующий скрипт читает все TSV-файлы корпуса, извлекает текст каждого документа и считает, сколько раз встречается каждое слово, игнорируя регистр. Затем слова сортируются по убыванию частоты, берутся топ-100 000 самых частых слов, и строится логарифмическая регрессия зависимости логарифма частоты от логарифма ранга слова. Скрипт вычисляет наклон линии регрессии и коэффициент детерминации R^2, а также визуализирует данные: синими точками показаны реальные частоты слов, а красной линией – гипотетический закон Ципфа $f = C/rankf$. Результаты регрессии (наклон ≈ 1.080 и $R^2 \approx 0.997$) показывают, что распределение слов очень хорошо

подчиняется степенному закону, с небольшим смещением: на протяжении почти всего диапазона рангов фактическая частота немного выше классического закона Ципфа, что видно на графике как точки чуть выше красной линии.



Стеммер

Stemmer.cpp реализует простой ручной стеммер для русского языка и демонстрирует его работу на словах, коротких словах, тексте, а затем применяет к файлам корпуса. Структура Stemmer хранит список суффиксов для отрезания, список исключений (частые служебные слова, предлоги, союзы), минимальную длину основы и методы для стемминга отдельных слов и целого текста. Функция stem переводит слово в нижний регистр, проверяет, является ли оно исключением, и если нет — обрезает подходящий суффикс, обеспечивая минимальную длину основы. Метод stem_text последовательно извлекает слова из текста, применяет стемминг и собирает результат. В main сначала тестируются отдельные слова, показывая, что, например, "книги", "книгу" и "книжный" приводятся к основе "книг", а "читать", "читает", "читали" — к "чит"; короткие слова вроде "и", "в", "на" остаются без изменений, что демонстрирует работу списка исключений; для текста "Я читаю интересную книгу о великих писателях и их произведениях" результат стемминга будет «я чита интерес интерес книг о велик писател и их произвед», эффективно нормализуя разные формы слов. После тестов скрипт проходит по всем файлам part_*.tsv корпуса, создает аналогичные файлы в папке literature_corpus_stemmed с уже стеммированными заголовками и текстами, сохраняя идентификаторы документов и заголовки, что позволяет использовать нормализованные данные для анализа или построения индекса

Булевый поиск и индекс

Boolean_Index.cpp реализует построение булева (инвертированного) индекса для корпуса текстов, поддерживая как обычные, так и стеммированные документы, и позволяет быстро находить, в каких документах встречается то или иное слово. Структура DocList хранит слово, массив документов, в которых оно встречается, текущий счетчик документов и емкость массива для динамического расширения. Функции add_entry и add_doc_to_word гарантируют, что каждое слово добавляется только один раз для каждого документа, а при необходимости массив документов расширяется через realloc. Основной цикл build_index проходит по всем TSV-файлам в указанной папке, читает строки, извлекает идентификатор документа и текст, разделяет текст на слова и добавляет их в индекс, одновременно ведя подсчет общего числа документов и записей «слово-документ». После обработки всех файлов данные сохраняются в JSON-формате, где ключ — слово, а значение — список документов, что позволяет использовать индекс для быстрого поиска и построения логических запросов. В конце выводится статистика: количество файлов, документов, уникальных слов, общих записей и среднее число документов на слово, а также примеры первых слов с привязкой к документам. В интерактивной части пользователь может выбрать, работать ли со стеммированными текстами или с исходными, и проверить конкретное слово в индексе, увидев, в каких документах оно встречается, что демонстрирует эффективность структуры для полнотекстового поиска

Boolean_Search.cpp реализует интерактивную систему булевого поиска по корпусу стеммированных текстов. Сначала загружается булев индекс из boolean_index_stemmed.json, где для каждого слова хранится список документов, в которых оно встречается (DocList), а также заголовки документов из папки ../literature_corpus_stemmed для отображения результатов. Пользователь вводит запрос с операторами AND (пересечение), OR (объединение), NOT (исключение) и скобками для группировки. Запрос разбивается на токены (tokenize_query), после чего происходит пошаговая обработка через evaluate и parse_expression, где выполняются соответствующие булевы операции над множествами документов, содержащими слова из запроса. Для каждого слова ищется его индекс через find_word, а операции AND, OR, NOT реализованы через функции set_and, set_or, set_not, которые пересекают, объединяют или исключают списки документов.. Программа интерактивная: пользователь видит токены запроса, количество найденных документов и первые 20 результатов, а также может вводить новые запросы до команды exit. Помимо этого результаты поиска сохраняются в search_results.txt с указанием времени, запроса, количества найденных документов и их заголовков

Выводы

В рамках проекта была разработана и реализована полнофункциональная система полнотекстового поиска, охватывающая весь цикл работы с текстовым корпусом: начиная с загрузки и очистки данных, нормализации текста, токенизации и фильтрации

нерелевантных элементов, включая опциональный стемминг для приведения слов к их базовым формам, и заканчивая построением как прямых, так и обратных индексов, обеспечивающих быстрый поиск документов по ключевым терминам; на базе этих индексов был реализован интерактивный модуль булевого поиска с поддержкой логических операций AND, OR, NOT и группировки через скобки, где запросы разбиваются на токены и последовательно обрабатываются для вычисления пересечений, объединений и исключений множеств документов, при этом результаты сопоставляются с заголовками и сохраняются с логированием времени и количества найденных документов; дополнительно проведён статистический анализ корпуса и распределения частот слов с проверкой соответствия законам Ципфа, что позволило оценить эффективность индексации и выявить характеристики текста, а также собрать сведения о количестве уникальных слов и среднем числе документов на слово; созданная архитектура обеспечивает надёжную и расширяемую основу для полнотекстового поиска, оставляя возможности для внедрения ранжирования, позиционных индексов и более удобных интерфейсов в будущих версиях системы.